Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №1
Варіант 9
На тему
“Основи функціонального програмування у Python”

Виконав:
Студент групи ФЕП-22
Обиход І.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати typing.Callable для параметризації обчислень.

Теоретичні відомості:

- Чиста функція: детермінована (однаковий вхід → однаковий вихід), без змін глобального стану/вхідних даних, без I/O усередині.
- Референтна прозорість: вираз можна замінити його значенням без зміни поведінки програми.
- Побічні ефекти: I/O (print, файл, мережа), мутація вхідних структур, використання/зміна глобалів, залежність від time, random, os.environ тощо.
- Підхід: відокремити чисте ядро (перетворення даних) від оболонки ефектів (читання/запис).

Завдання (варіант 9 - Прокат автомобілів):

Вхідні дані: клас авто, кількість днів, денна ставка, статус бронювання. Потрібно обчислити базову ціну прокату, знижку за тривалість і страховий збір. Політики знижки та страхування мають передаватися як функції (typing.Callable); вхідні записи не змінюються, ядро не містить глобального стану та I/O.

Хід роботи

Під час виконання лабораторної роботи було створено програмне рішення для розрахунку вартості прокату автомобілів. Архітектура проєкту побудована на засадах функціонального програмування з чітким відокремленням обчислювальної логіки від побічних ефектів. Проєкт складається з чистого ядра core.py, оболонки побічних ефектів app.py, тестів test/test_core.py та файлу requirements.txt зі списком інструментів (pytest, mypy, black, ruff).

Вихідний імперативний варіант мав типові проблеми: глобальні константи ставок знижки і страхування усередині обчислень, виклик print у циклі, мутацію вхідних словників через r["total"] = ... та накопичувальні змінні. У ході роботи код було переписано так, щоб жодна з цих проблем не залишилась.

Завдання 1. У модулі core.py описано типи предметної області Rental і Report (TypedDict) та реалізовано чисті функції: base_price (кількість днів × денна ставка), is_confirmed (перевірка статусу), duration_discount_rate (ставка знижки

за порогами тривалості), insurance_rate_for (ставка страховки за класом авто), with_pricing (створює новий запис із полями base_price, discount, insurance, total, не змінюючи вхідний) та summarize (формує звіт). Головна функція process_rentals_pure приймає усі параметри явно: мінімальну кількість днів, пороги знижки та ставки страхування. Вона відбирає підтверджені прокати, обчислює ціну і повертає новий словник із кількістю, сумарним доходом та новими записами.

Розрахунок виконується за формулою: base = days × daily_rate; discount = base × rate(days), де ставка становить 10 % від 7 днів, 15 % від 14 і 25 % від 30 днів; insurance = (base − discount) × rate(car_class), де economy 5 %, comfort 8 %, business 12 %, premium 18 %; total = base − discount + insurance.

Завдання 2. Щоб зробити систему гнучкою, обчислення параметризовано за допомогою typing.Callable. Визначено типи політик AcceptFn (фільтр), DiscountFn (знижка) та InsuranceFn (страховий збір). Фабрика make_processor(accept=..., apply_discount=..., insurance_fee=...) повертає налаштований обробник, у який будь-яку політику можна підставити без зміни коду. Реалізовано фабрики політик make_min_days_filter, make_duration_discount, make_class_insurance, make_per_day_insurance та lift_price_fn, що підіймає просту функцію ціни до рівня політики знижки.

Завдання 3. Усі побічні ефекти винесено в app.py: там зберігаються тестові дані, а друк звіту виконує render_report, яка лише споживає готовий результат. Поточний час не викликається в ядрі: функції stamp_total і stamp_report приймають постачальник часу NowFn, а time.time передається з оболонки, тому ядро лишається чистим і тестується з фейковим годинником.

Додаткові вправи. Реалізовано compose та pipe для побудови знижки з кількох маленьких функцій (make_percent_off, make_coupon), фабрику множників make_multiplier, а також функції mark_cancelled і totals_by_class, які замість мутації списку повертають нові колекції через list comprehension.

Надійність реалізованого підходу доведено автоматизованими тестами pytest (20 тестів у файлі test/test_core.py). Вони перевіряють референтну прозорість, відсутність мутації вхідних даних, коректність числових результатів, замінюваність Callable-політик, інжекцію часу та додаткові функції. Перевірка mypy у строгому режимі не виявила помилок типізації, а black і ruff підтвердили відповідність коду стандартам форматування.

Файл core.py

core.py рядки 1-62

```python
from __future__ import annotations

from collections.abc import Callable, Iterable, Mapping, Sequence
from typing import TypedDict, TypeVar


class Rental(TypedDict, total=False):
    id: int
    car_class: str
    days: int
    daily_rate: float
    status: str
    base_price: float
    discount: float
    insurance: float
    total: float


class Report(TypedDict):
    count: int
    revenue: float
    rentals: list[Rental]


PriceFn = Callable[[float], float]
DiscountFn = Callable[[Rental, float], float]
InsuranceFn = Callable[[Rental, float], float]
AcceptFn = Callable[[Rental], bool]
NowFn = Callable[[], float]
Processor = Callable[[Iterable[Rental]], Report]


def base_price(rental: Rental) -> float:
    return rental["days"] * rental["daily_rate"]


def is_confirmed(rental: Rental) -> bool:
    return rental["status"] == "confirmed"


def duration_discount_rate(days: int, tiers: Sequence[tuple[int, float]]) -> float:
    reached = [rate for min_days, rate in tiers if days >= min_days]
    return max(reached, default=0.0)


def insurance_rate_for(
    car_class: str, rates: Mapping[str, float], default: float
) -> float:
    return rates.get(car_class, default)


def with_pricing(
    rental: Rental, *, base: float, discount: float, insurance: float
) -> Rental:
    return {
        **rental,
        "base_price": base,
        "discount": discount,
        "insurance": insurance,
        "total": base - discount + insurance,
    }
```

core.py рядки 63-123

```python
def summarize(rentals: Sequence[Rental]) -> Report:
    return {
        "count": len(rentals),
        "revenue": sum(r["total"] for r in rentals),
        "rentals": list(rentals),
    }


def process_rentals_pure(
    rentals: Iterable[Rental],
    *,
    min_days: int,
    discount_tiers: Sequence[tuple[int, float]],
    insurance_rates: Mapping[str, float],
    default_insurance_rate: float,
) -> Report:
    confirmed = (r for r in rentals if is_confirmed(r))
    qualified = (r for r in confirmed if r["days"] >= min_days)

    priced: list[Rental] = []
    for r in qualified:
        base = base_price(r)
        discount = base * duration_discount_rate(r["days"], discount_tiers)
        discounted = base - discount
        insurance = discounted * insurance_rate_for(
            r["car_class"], insurance_rates, default_insurance_rate
        )
        priced.append(
            with_pricing(r, base=base, discount=discount, insurance=insurance)
        )

    return summarize(priced)


def make_processor(
    *,
    accept: AcceptFn,
    apply_discount: DiscountFn,
    insurance_fee: InsuranceFn,
) -> Processor:

    def price_one(rental: Rental) -> Rental:
        base = base_price(rental)
        discounted = apply_discount(rental, base)
        fee = insurance_fee(rental, discounted)
        return with_pricing(
            rental, base=base, discount=base - discounted, insurance=fee
        )

    def process(rentals: Iterable[Rental]) -> Report:
        priced = [price_one(r) for r in rentals if accept(r)]
        return summarize(priced)

    return process


def make_min_days_filter(min_days: int) -> AcceptFn:
    return lambda r: is_confirmed(r) and r["days"] >= min_days
```

core.py рядки 124-182

```python
def make_duration_discount(tiers: Sequence[tuple[int, float]]) -> DiscountFn:
    frozen = tuple(tiers)
    return lambda r, price: price * (1 - duration_discount_rate(r["days"], frozen))


def make_class_insurance(
    rates: Mapping[str, float], default: float = 0.0
) -> InsuranceFn:
    frozen = dict(rates)
    return lambda r, price: price * insurance_rate_for(r["car_class"], frozen, default)


def make_per_day_insurance(fee_per_day: float) -> InsuranceFn:
    return lambda r, _price: r["days"] * fee_per_day


def lift_price_fn(fn: PriceFn) -> DiscountFn:
    return lambda _r, price: fn(price)


def stamp_total(total: float, now: NowFn) -> tuple[float, float]:
    return total, now()


def stamp_report(report: Report, now: NowFn) -> dict[str, object]:
    return {**report, "generated_at": now()}


A = TypeVar("A")
B = TypeVar("B")
C = TypeVar("C")


def compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]:
    return lambda x: f(g(x))


def pipe(*fns: PriceFn) -> PriceFn:
    def identity(x: float) -> float:
        return x

    result: PriceFn = identity
    for fn in fns:
        result = compose(fn, result)
    return result


def make_multiplier(k: int) -> Callable[[int], int]:
    return lambda x: x * k


def make_percent_off(percent: float) -> PriceFn:
    return lambda price: price * (1 - percent / 100)


def make_coupon(amount: float) -> PriceFn:
    return lambda price: max(0.0, price - amount)
```

core.py рядки 183-195

```python
def mark_cancelled(rentals: Sequence[Rental], ids: Iterable[int]) -> list[Rental]:
    to_cancel = frozenset(ids)
    return [
        {**r, "status": "cancelled"} if r["id"] in to_cancel else r for r in rentals
    ]


def totals_by_class(rentals: Sequence[Rental]) -> dict[str, float]:
    classes = sorted({r["car_class"] for r in rentals})
    return {
        cls: sum(r["total"] for r in rentals if r["car_class"] == cls)
        for cls in classes
    }
```

Файл app.py

app.py рядки 1-62

```python
from __future__ import annotations

import time
from collections.abc import Sequence
from datetime import datetime, timezone

from core import (
    Rental,
    Report,
    lift_price_fn,
    make_class_insurance,
    make_coupon,
    make_duration_discount,
    make_min_days_filter,
    make_per_day_insurance,
    make_percent_off,
    make_processor,
    pipe,
    process_rentals_pure,
    stamp_report,
    totals_by_class,
)

RENTALS: list[Rental] = [
    {
        "id": 1,
        "car_class": "economy",
        "days": 3,
        "daily_rate": 800.0,
        "status": "confirmed",
    },
    {
        "id": 2,
        "car_class": "comfort",
        "days": 7,
        "daily_rate": 1200.0,
        "status": "confirmed",
    },
    {
        "id": 3,
        "car_class": "business",
        "days": 1,
        "daily_rate": 2500.0,
        "status": "confirmed",
    },
    {
        "id": 4,
        "car_class": "premium",
        "days": 14,
        "daily_rate": 4000.0,
        "status": "confirmed",
    },
    {
        "id": 5,
        "car_class": "economy",
        "days": 10,
        "daily_rate": 750.0,
        "status": "cancelled",
    },
    {
        "id": 6,
        "car_class": "comfort",
```

app.py рядки 63-119

```
        "days": 30,
        "daily_rate": 1100.0,
        "status": "confirmed",
    },
    {
        "id": 7,
        "car_class": "business",
        "days": 5,
        "daily_rate": 2300.0,
        "status": "pending",
    },
    {
        "id": 8,
        "car_class": "suv",
        "days": 4,
        "daily_rate": 1900.0,
        "status": "confirmed",
    },
]

DISCOUNT_TIERS: Sequence[tuple[int, float]] = ((7, 0.10), (14, 0.15), (30, 0.25))
INSURANCE_RATES = {
    "economy": 0.05,
    "comfort": 0.08,
    "business": 0.12,
    "premium": 0.18,
}


def render_report(title: str, report: Report) -> None:
    print(f"=== {title} ===")
    print(
        f"{'ID':>4} {'Клас':<10} {'Днів':>4} {'База':>10} {'Знижка':>9} "
        f"{'Страх.':>9} {'Разом':>10}"
    )
    for r in report["rentals"]:
        print(
            f"{r['id']:>4} {r['car_class']:<10} {r['days']:>4} "
            f"{r['base_price']:>10.2f} {r['discount']:>9.2f} "
            f"{r['insurance']:>9.2f} {r['total']:>10.2f}"
        )
    print(f"Прийнято прокатів: {report['count']}")
    print(f"Дохід:             {report['revenue']:.2f}")
    by_class = totals_by_class(report["rentals"])
    for cls, total in by_class.items():
        print(f"  {cls:<10} {total:>10.2f}")
    print()


def render_stamped(report: Report) -> None:
    stamped = stamp_report(report, time.time)
    ts = float(str(stamped["generated_at"]))
    when = datetime.fromtimestamp(ts, tz=timezone.utc).isoformat(timespec="seconds")
    print(f"Звіт сформовано: {when} (UTC)")
    print()
```

app.py рядки 120-149

```python
def run(rentals: Sequence[Rental]) -> None:
    basic = process_rentals_pure(
        rentals,
        min_days=2,
        discount_tiers=DISCOUNT_TIERS,
        insurance_rates=INSURANCE_RATES,
        default_insurance_rate=0.10,
    )
    render_report("Завдання 1: process_rentals_pure", basic)

    standard = make_processor(
        accept=make_min_days_filter(2),
        apply_discount=make_duration_discount(DISCOUNT_TIERS),
        insurance_fee=make_class_insurance(INSURANCE_RATES, default=0.10),
    )
    render_report("Завдання 2: make_processor (стандартні політики)", standard(rentals))

    promo_discount = pipe(make_percent_off(5), make_coupon(50.0))
    promo = make_processor(
        accept=make_min_days_filter(1),
        apply_discount=lift_price_fn(promo_discount),
        insurance_fee=make_per_day_insurance(15.0),
    )
    render_report("Завдання 2: make_processor (акційні політики)", promo(rentals))

    render_stamped(basic)


if __name__ == "__main__":
    run(RENTALS)
```

Файл test_core.py

test_core.py рядки 1-62

```python
from __future__ import annotations

from collections.abc import Callable
from copy import deepcopy

import pytest

from core import (
    DiscountFn,
    InsuranceFn,
    Rental,
    Report,
    base_price,
    compose,
    duration_discount_rate,
    insurance_rate_for,
    lift_price_fn,
    make_class_insurance,
    make_coupon,
    make_duration_discount,
    make_min_days_filter,
    make_multiplier,
    make_per_day_insurance,
    make_percent_off,
    make_processor,
    mark_cancelled,
    pipe,
    process_rentals_pure,
    stamp_report,
    stamp_total,
    totals_by_class,
    with_pricing,
)

TIERS = ((7, 0.10), (30, 0.25))
RATES = {"economy": 0.05, "comfort": 0.08, "business": 0.12, "premium": 0.18}


@pytest.fixture
def sample_rentals() -> list[Rental]:
    return [
        {
            "id": 1,
            "car_class": "economy",
            "days": 3,
            "daily_rate": 100.0,
            "status": "confirmed",
        },
        {
            "id": 2,
            "car_class": "comfort",
            "days": 7,
            "daily_rate": 200.0,
            "status": "confirmed",
        },
        {
            "id": 3,
            "car_class": "business",
            "days": 1,
            "daily_rate": 500.0,
            "status": "confirmed",
        },
```

test_core.py рядки 63-115

```python
        {
            "id": 4,
            "car_class": "premium",
            "days": 30,
            "daily_rate": 400.0,
            "status": "confirmed",
        },
        {
            "id": 5,
            "car_class": "economy",
            "days": 10,
            "daily_rate": 100.0,
            "status": "cancelled",
        },
        {
            "id": 6,
            "car_class": "suv",
            "days": 2,
            "daily_rate": 300.0,
            "status": "pending",
        },
    ]


def run_pure(rentals: list[Rental]) -> Report:
    return process_rentals_pure(
        rentals,
        min_days=2,
        discount_tiers=TIERS,
        insurance_rates=RATES,
        default_insurance_rate=0.10,
    )


def test_referential_transparency(sample_rentals: list[Rental]) -> None:
    r1 = run_pure(sample_rentals)
    r2 = run_pure(sample_rentals)
    assert r1 == r2


def test_no_mutation(sample_rentals: list[Rental]) -> None:
    original = deepcopy(sample_rentals)
    run_pure(sample_rentals)
    assert sample_rentals == original


def test_result_records_are_new_objects(sample_rentals: list[Rental]) -> None:
    result = run_pure(sample_rentals)
    inputs = {id(r) for r in sample_rentals}
    assert all(id(r) not in inputs for r in result["rentals"])
    assert all("total" not in r for r in sample_rentals)
```

test_core.py рядки 116-176

```python
def test_processor_no_mutation(sample_rentals: list[Rental]) -> None:
    original = deepcopy(sample_rentals)
    processor = make_processor(
        accept=make_min_days_filter(0),
        apply_discount=make_duration_discount(TIERS),
        insurance_fee=make_class_insurance(RATES, default=0.1),
    )
    processor(sample_rentals)
    assert sample_rentals == original


def test_base_price() -> None:
    rental: Rental = {
        "id": 1,
        "car_class": "economy",
        "days": 3,
        "daily_rate": 100.0,
        "status": "confirmed",
    }
    assert base_price(rental) == 300.0


def test_duration_discount_rate() -> None:
    assert duration_discount_rate(3, TIERS) == 0.0
    assert duration_discount_rate(7, TIERS) == 0.10
    assert duration_discount_rate(29, TIERS) == 0.10
    assert duration_discount_rate(30, TIERS) == 0.25
    assert duration_discount_rate(100, ()) == 0.0


def test_insurance_rate_for() -> None:
    assert insurance_rate_for("premium", RATES, 0.1) == 0.18
    assert insurance_rate_for("unknown", RATES, 0.1) == 0.1


def test_with_pricing_returns_new_dict() -> None:
    rental: Rental = {
        "id": 1,
        "car_class": "economy",
        "days": 3,
        "daily_rate": 100.0,
        "status": "confirmed",
    }
    priced = with_pricing(rental, base=300.0, discount=30.0, insurance=13.5)
    assert priced is not rental
    assert "total" not in rental
    assert priced["total"] == pytest.approx(283.5)
    assert priced["id"] == 1


def test_process_rentals_pure_values(sample_rentals: list[Rental]) -> None:
    result = run_pure(sample_rentals)
    assert [r["id"] for r in result["rentals"]] == [1, 2, 4]
    assert result["count"] == 3

    by_id = {r["id"]: r for r in result["rentals"]}
    assert by_id[1]["total"] == pytest.approx(315.0)
    assert by_id[2]["discount"] == pytest.approx(140.0)
    assert by_id[2]["total"] == pytest.approx(1360.8)
    assert by_id[4]["total"] == pytest.approx(10620.0)
```

test_core.py рядки 177-238

```python
    assert result["revenue"] == pytest.approx(315.0 + 1360.8 + 10620.0)


def test_callable_policies(sample_rentals: list[Rental]) -> None:
    accept = lambda r: r["status"] == "confirmed" and base_price(r) >= 500
    apply_discount: DiscountFn = lambda r, p: p * 0.9
    insurance_fee: InsuranceFn = lambda r, p: p * 0.2

    processor = make_processor(
        accept=accept, apply_discount=apply_discount, insurance_fee=insurance_fee
    )
    result = processor(sample_rentals)

    assert [r["id"] for r in result["rentals"]] == [2, 3, 4]
    expected = sum(b * 0.9 * 1.2 for b in (1400.0, 500.0, 12000.0))
    assert result["revenue"] == pytest.approx(expected)


def test_processor_matches_pure_version(sample_rentals: list[Rental]) -> None:
    processor = make_processor(
        accept=make_min_days_filter(2),
        apply_discount=make_duration_discount(TIERS),
        insurance_fee=make_class_insurance(RATES, default=0.10),
    )
    assert processor(sample_rentals) == run_pure(sample_rentals)


def test_swapping_policy_changes_only_that_part(sample_rentals: list[Rental]) -> None:
    accept = make_min_days_filter(2)
    discount = make_duration_discount(TIERS)
    percent = make_processor(
        accept=accept,
        apply_discount=discount,
        insurance_fee=make_class_insurance(RATES, 0.1),
    )
    per_day = make_processor(
        accept=accept,
        apply_discount=discount,
        insurance_fee=make_per_day_insurance(10.0),
    )

    p, d = percent(sample_rentals), per_day(sample_rentals)
    assert [r["id"] for r in p["rentals"]] == [r["id"] for r in d["rentals"]]
    for a, b in zip(p["rentals"], d["rentals"], strict=True):
        assert a["discount"] == pytest.approx(b["discount"])
        assert b["insurance"] == pytest.approx(b["days"] * 10.0)


def test_make_duration_discount_is_isolated_from_later_changes() -> None:
    tiers = [(7, 0.5)]
    policy = make_duration_discount(tiers)
    tiers.append((1, 0.99))
    rental: Rental = {
        "id": 1,
        "car_class": "economy",
        "days": 2,
        "daily_rate": 10.0,
        "status": "confirmed",
    }
    assert policy(rental, 20.0) == 20.0
```

test_core.py рядки 239-295

```python
def test_stamp_total_uses_injected_clock() -> None:
    fake_now = lambda: 1_700_000_000.0
    assert stamp_total(99.0, fake_now) == (99.0, 1_700_000_000.0)
    assert stamp_total(99.0, fake_now) == stamp_total(99.0, fake_now)


def test_stamp_report_does_not_mutate(sample_rentals: list[Rental]) -> None:
    report = run_pure(sample_rentals)
    before = deepcopy(report)
    stamped = stamp_report(report, lambda: 1.0)
    assert report == before
    assert stamped["generated_at"] == 1.0
    assert "generated_at" not in report


def test_compose() -> None:
    inc: Callable[[int], int] = lambda x: x + 1
    dbl: Callable[[int], int] = lambda x: x * 2
    assert compose(inc, dbl)(5) == 11
    assert compose(dbl, inc)(5) == 12


def test_pipe_builds_discount_from_small_functions() -> None:
    discount = pipe(make_percent_off(10), make_coupon(20.0))
    assert discount(200.0) == pytest.approx(160.0)
    assert make_coupon(500.0)(100.0) == 0.0

    rental: Rental = {
        "id": 1,
        "car_class": "economy",
        "days": 1,
        "daily_rate": 200.0,
        "status": "confirmed",
    }
    assert lift_price_fn(discount)(rental, 200.0) == pytest.approx(160.0)


def test_make_multiplier() -> None:
    triple = make_multiplier(3)
    assert triple(5) == 15
    assert [make_multiplier(k)(2) for k in (1, 2, 3)] == [2, 4, 6]


def test_mark_cancelled_returns_new_list(sample_rentals: list[Rental]) -> None:
    original = deepcopy(sample_rentals)
    updated = mark_cancelled(sample_rentals, [1, 2])
    assert sample_rentals == original
    assert updated is not sample_rentals
    assert [r["status"] for r in updated[:2]] == ["cancelled", "cancelled"]
    assert updated[2] is sample_rentals[2]


def test_totals_by_class(sample_rentals: list[Rental]) -> None:
    result = run_pure(sample_rentals)
    totals = totals_by_class(result["rentals"])
    assert set(totals) == {"economy", "comfort", "premium"}
    assert sum(totals.values()) == pytest.approx(result["revenue"])
```

Запуск програми

```
Windows PowerShell

PS D:\IPZ2kurs\func\lab1> py app.py
=== Завдання 1: process_rentals_pure ===
  ID Клас       Днів       База    Знижка    Страх.      Разом
   1 economy       3    2400.00      0.00    120.00    2520.00
   2 comfort       7    8400.00    840.00    604.80    8164.80
   4 premium      14   56000.00   8400.00   8568.00   56168.00
   6 comfort      30   33000.00   8250.00   1980.00   26730.00
   8 suv           4    7600.00      0.00    760.00    8360.00
Прийнято прокатів: 5
Дохід:            101942.80
  comfort      34894.80
  economy       2520.00
  premium      56168.00
  suv           8360.00

=== Завдання 2: make_processor (стандартні політики) ===
  ID Клас       Днів       База    Знижка    Страх.      Разом
   1 economy       3    2400.00      0.00    120.00    2520.00
   2 comfort       7    8400.00    840.00    604.80    8164.80
   4 premium      14   56000.00   8400.00   8568.00   56168.00
   6 comfort      30   33000.00   8250.00   1980.00   26730.00
   8 suv           4    7600.00      0.00    760.00    8360.00
Прийнято прокатів: 5
Дохід:            101942.80
  comfort      34894.80
  economy       2520.00
  premium      56168.00
  suv           8360.00

=== Завдання 2: make_processor (акційні політики) ===
  ID Клас       Днів       База    Знижка    Страх.      Разом
   1 economy       3    2400.00    170.00     45.00    2275.00
   2 comfort       7    8400.00    470.00    105.00    8035.00
   3 business      1    2500.00    175.00     15.00    2340.00
   4 premium      14   56000.00   2850.00    210.00   53360.00
   6 comfort      30   33000.00   1700.00    450.00   31750.00
   8 suv           4    7600.00    430.00     60.00    7230.00
Прийнято прокатів: 6
Дохід:            104990.00
  business      2340.00
  comfort      39785.00
  economy       2275.00
  premium      53360.00
  suv           7230.00

Звіт сформовано: 2026-10-22T20:16:58+00:00 (UTC)
```

Рис. 1. Вивід програми app.py: звіти за двома наборами політик і позначка часу

Виконання тестів

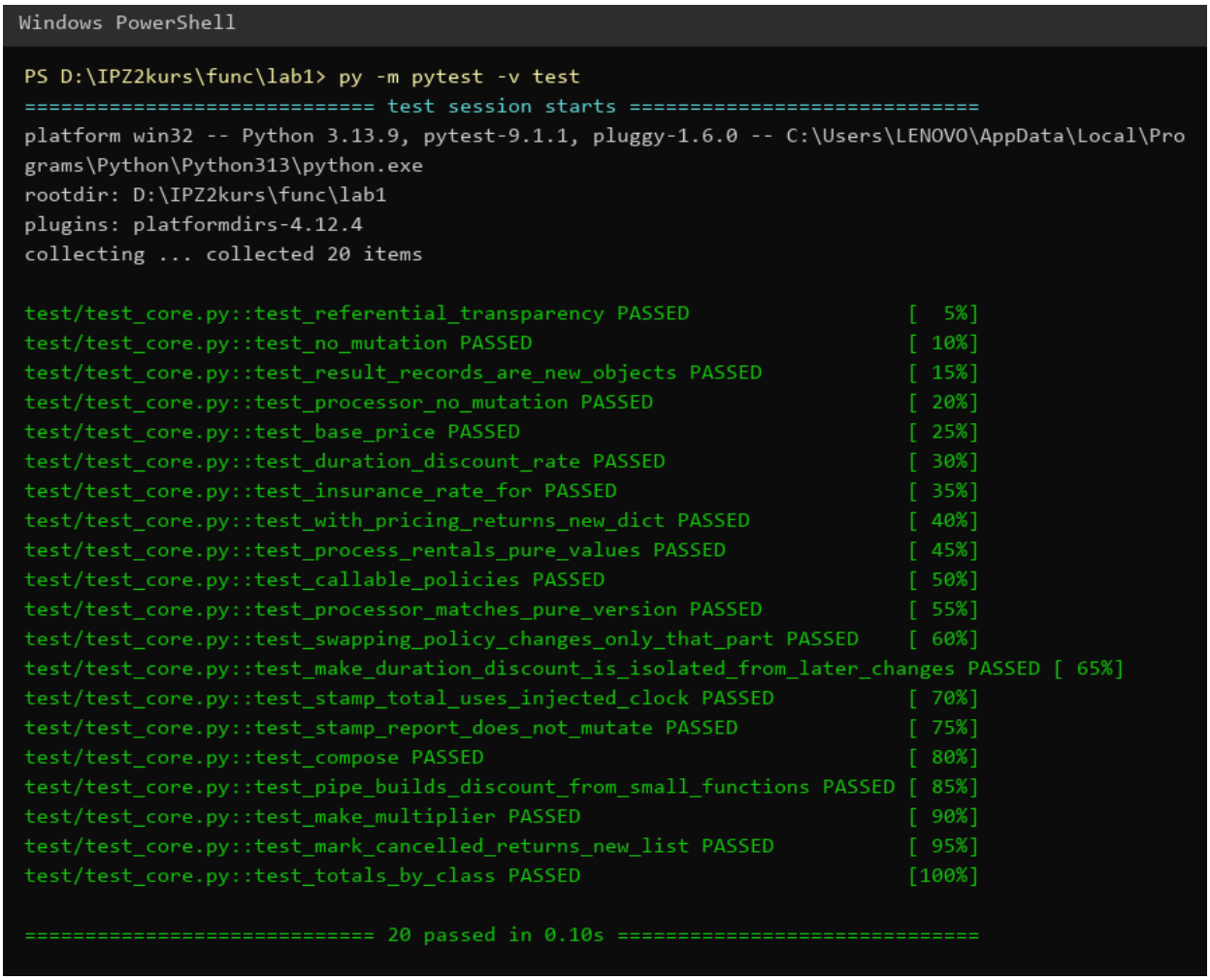


Рис. 2. Результат запуску pytest: 20 тестів пройдено

Перевірка типів і стилю

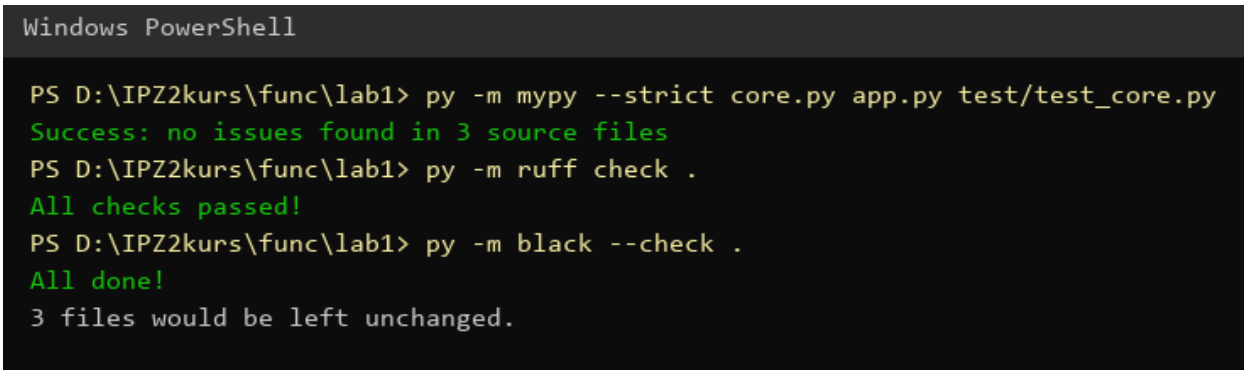


Рис. 3. mypy (strict), ruff та black без зауважень

Висновок:

Під час виконання лабораторної роботи початковий імперативний код на Python було успішно переписано у функціональному стилі. На практиці опрацьовано створення чистих функцій предметної області та відокремлення обчислень від введення-виведення: усі print і звернення до системного часу

зосереджено в оболонці app.py, а ядро core.py не має побічних ефектів. Завдяки typing.Callable політики знижки, страхування та фільтрації передаються як звичайні аргументи, а фабрика make_processor дозволяє збирати обробник із будь-якого їх набору. Розроблений код не використовує глобальних змінних і не змінює вхідних даних, що робить його референтно прозорим, передбачуваним і легким для тестування, що підтверджено 20 pytest-тестами та перевіркою mypy.